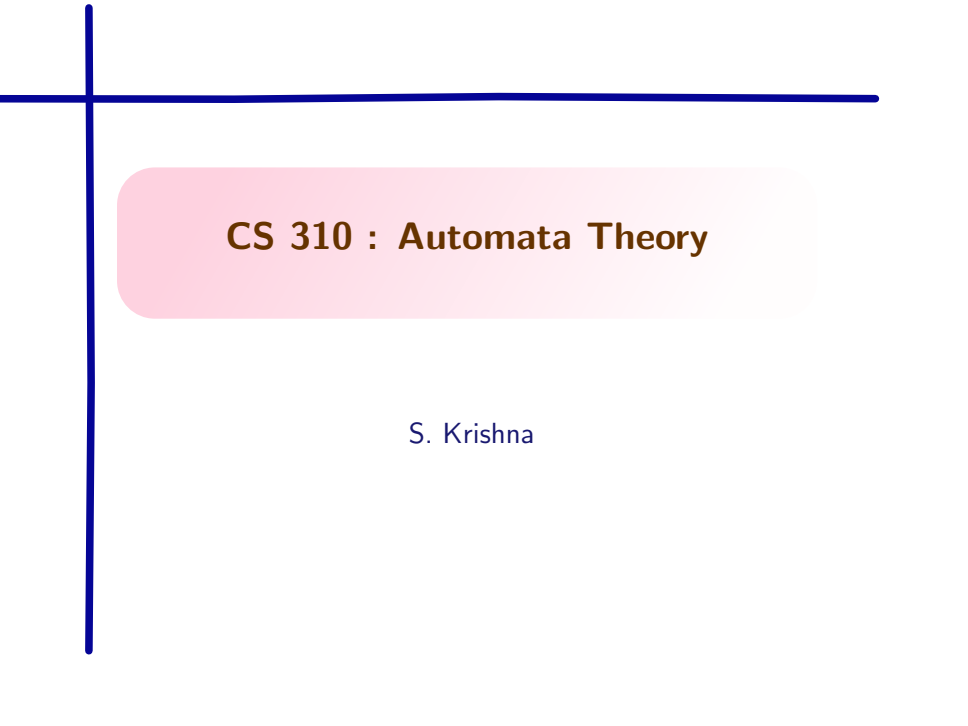


A decorative blue crosshair consisting of a vertical line and a horizontal line intersecting at the top-left corner of the slide.

CS 310 : Automata Theory

S. Krishna

Recall: The family L_n

Let $\Sigma = \{0, 1\}$. Consider the family $L_n = \{ww \mid w \in \Sigma^n\}$ of languages. Any NFA accepting L_n needs $\geq 2^n$ states.

Recall: The family L_n

Let $\Sigma = \{0, 1\}$. Consider the family $L_n = \{ww \mid w \in \Sigma^n\}$ of languages. Any NFA accepting L_n needs $\geq 2^n$ states.

- Proof idea: for each $w \in \Sigma^n$, let q_w be any state reached after reading the first half of an accepting run on ww .

Recall: The family L_n

Let $\Sigma = \{0, 1\}$. Consider the family $L_n = \{ww \mid w \in \Sigma^n\}$ of languages. Any NFA accepting L_n needs $\geq 2^n$ states.

- ▶ Proof idea: for each $w \in \Sigma^n$, let q_w be any state reached after reading the first half of an accepting run on ww .
- ▶ If fewer than 2^n states, then by PHP, $\exists w \neq w'$ with $q_w = q_{w'}$.
- ▶ Then $w'w$ is accepted with $w' \neq w$

Recall: The family L_n

Let $\Sigma = \{0, 1\}$. Consider the family $L_n = \{ww \mid w \in \Sigma^n\}$ of languages. Any NFA accepting L_n needs $\geq 2^n$ states.

- ▶ Proof idea: for each $w \in \Sigma^n$, let q_w be any state reached after reading the first half of an accepting run on ww .
- ▶ If fewer than 2^n states, then by PHP, $\exists w \neq w'$ with $q_w = q_{w'}$.
- ▶ Then $w'w$ is accepted with $w' \neq w$
- ▶ Can we have a better argument than using PHP?

Recall: The family L_n

Let $\Sigma = \{0, 1\}$. Consider the family $L_n = \{ww \mid w \in \Sigma^n\}$ of languages. Any NFA accepting L_n needs $\geq 2^n$ states.

- ▶ Proof idea: for each $w \in \Sigma^n$, let q_w be any state reached after reading the first half of an accepting run on ww .
- ▶ If fewer than 2^n states, then by PHP, $\exists w \neq w'$ with $q_w = q_{w'}$.
- ▶ Then $w'w$ is accepted with $w' \neq w$
- ▶ Can we have a better argument than using PHP?
- ▶ Do we have a canonical minimal NFA like in the case of DFA?

No canonical minimal NFA

- ▶ The Myhill-Nerode theorem gives a canonical minimal DFA whose size is exactly the number of equivalence classes induced by $\sim_L$

No canonical minimal NFA

- ▶ The Myhill-Nerode theorem gives a canonical minimal DFA whose size is exactly the number of equivalence classes induced by $\sim_L$
- ▶ No such theorem is known for NFAs
- ▶ No canonical exists for minimal NFA, and NFA minimization is a hard problem.

No canonical minimal NFA

- ▶ The Myhill-Nerode theorem gives a canonical minimal DFA whose size is exactly the number of equivalence classes induced by $\sim_L$
- ▶ No such theorem is known for NFAs
- ▶ No canonical exists for minimal NFA, and NFA minimization is a hard problem.
- ▶ So a PHP argument is all we could use

No canonical minimal NFA

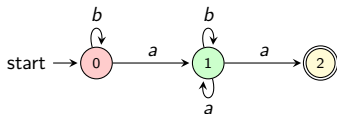
Minimal NFA (in states and transitions)

$$L = \{w \mid w \text{ has } \geq 2 \text{ occurrences of } a, \text{ and ends in } a\} = \Sigma^* a \Sigma^* a.$$

No canonical minimal NFA

Minimal NFA (in states and transitions)

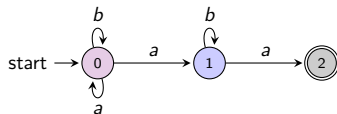
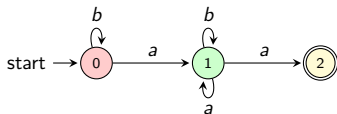
$L = \{w \mid w \text{ has } \geq 2 \text{ occurrences of } a, \text{ and ends in } a\} = \Sigma^* a \Sigma^* a.$



No canonical minimal NFA

Minimal NFA (in states and transitions)

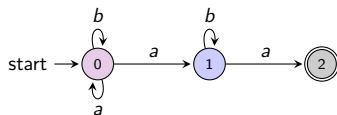
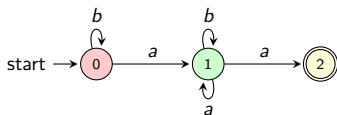
$L = \{w \mid w \text{ has } \geq 2 \text{ occurrences of } a, \text{ and ends in } a\} = \Sigma^* a \Sigma^* a.$



No canonical minimal NFA

Minimal NFA (in states and transitions)

$L = \{w \mid w \text{ has } \geq 2 \text{ occurrences of } a, \text{ and ends in } a\} = \Sigma^* a \Sigma^* a.$



Are these both minimal? Are these isomorphic?

Succinct Automata for L_n ?

We know that **angelic** NFA for L_n are huge

Succinct Automata for L_n ?

We know that **angelic** NFA for L_n are huge

Recall : Devilish NFA

- ▶ $(Q, \Sigma, \delta, q_0, F)$, $\delta : Q \times \Sigma \rightarrow 2^Q$.
- ▶ Let $\hat{\delta}(q_0, w)$ be the set of *all* states reachable after reading w .
- ▶ $L(A) = \{w \mid \emptyset \neq \hat{\delta}(q_0, w) \subseteq F\}$
- ▶ Also called **Universal automata (UFA)**
- ▶ We know angelic NFA and UFA both accept regular languages
- ▶ But one of them can be **exponentially more succinct** than the other

UFA for L_n

- ▶ $w = w_1 \cdots w_{2n} \in L_n$ iff $w_i = w_{n+i}$ for every $i = 1, \dots, n$.

UFA for L_n

- ▶ $w = w_1 \cdots w_{2n} \in L_n$ iff $w_i = w_{n+i}$ for every $i = 1, \dots, n$.
- ▶ Universal check : Can we check in parallel over the first n symbols?

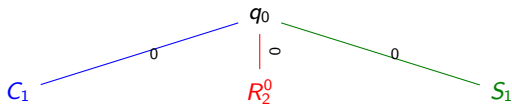
UFA for L_n

- ▶ $w = w_1 \cdots w_{2n} \in L_n$ iff $w_i = w_{n+i}$ for every $i = 1, \dots, n$.
- ▶ Universal check : Can we check in parallel over the first n symbols?
- ▶ Branch into n threads, one per i ; *all* must check $w_i = w_{n+i}$.

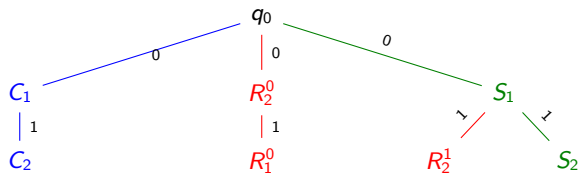
Two independent checks

- ▶ a length check (is $|w| = 2n$?),
 - ▶ n threads to match the first n positions and *stop tracking position* once their own comparison is made.
-
- ▶ From the initial state q_0 , branch into the length check and position matching check

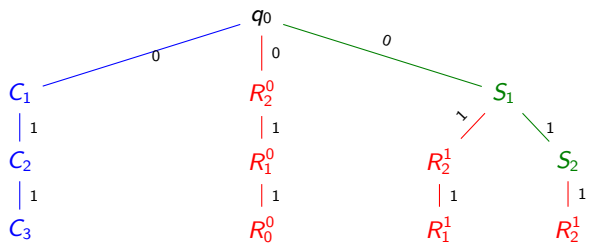
$$n = 3, w = 011, ww = 011011$$



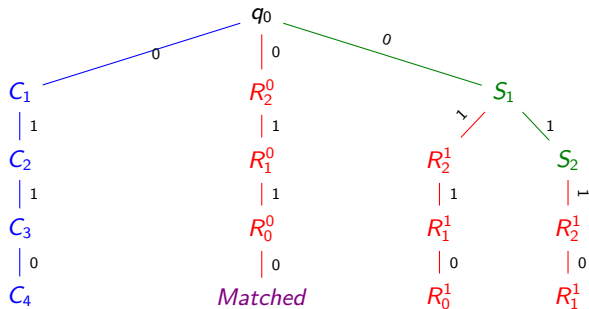
$n = 3$, $w = 011$, $ww = 011011$



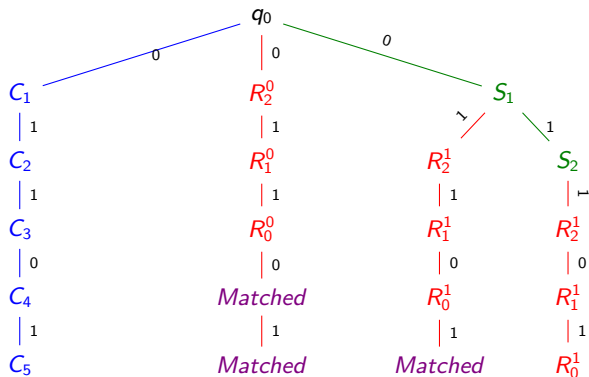
$n = 3$, $w = 011$, $ww = 011011$



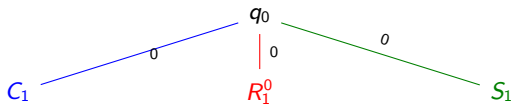
$n = 3, w = 011, ww = 011011$



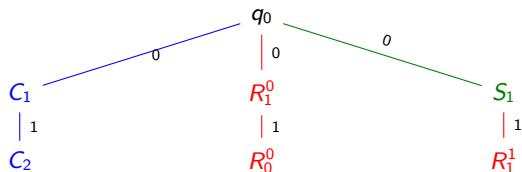
$n = 3, w = 011, ww = 011011$



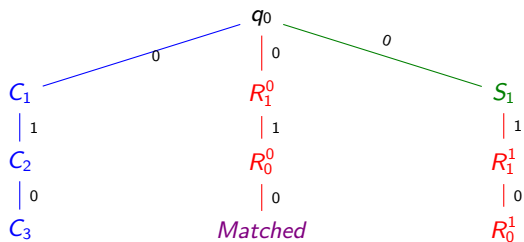
$n = 2$, $w = 01$, $ww' = 0100$ (a failed attempt)



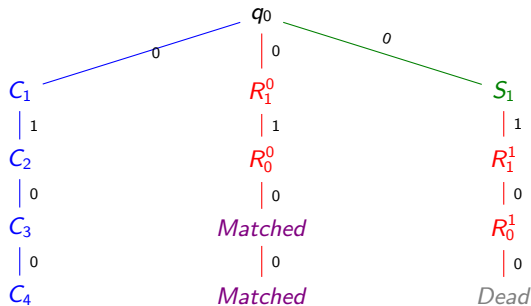
$n = 2$, $w = 01$, $ww' = 0100$ (a failed attempt)



$n = 2$, $w = 01$, $ww' = 0100$ (a failed attempt)



$n = 2$, $w = 01$, $ww' = 0100$ (a failed attempt)



States: $O(n)$ total

- ▶ Initial state q_0
- ▶ C_p ($1 \leq p \leq 2n$): length-checker

States: $O(n)$ total

- ▶ Initial state q_0
- ▶ C_p ($1 \leq p \leq 2n$): length-checker
- ▶ S_p ($1 \leq p \leq n - 1$): continue

States: $O(n)$ total

- ▶ Initial state q_0
- ▶ C_p ($1 \leq p \leq 2n$): length-checker
- ▶ S_p ($1 \leq p \leq n - 1$): continue
- ▶ R_k^b ($0 \leq k \leq n - 1$, $b \in \{0, 1\}$): remember b , skip next k symbols.

States: $O(n)$ total

- ▶ Initial state q_0
- ▶ C_p ($1 \leq p \leq 2n$): length-checker
- ▶ S_p ($1 \leq p \leq n - 1$): continue
- ▶ R_k^b ($0 \leq k \leq n - 1$, $b \in \{0, 1\}$): remember b , skip next k symbols.
- ▶ *Matched or Dead*

Total: $1 + 2n + 1 + n - 1 + 2n + 2 = 5n + 3$ states.

Transitions

- ▶ Length-checker: $C_p \xrightarrow{a} C_{p+1}$, oblivious to symbol value; C_{2n} is the only accepting state in this set.

Transitions

- ▶ Length-checker: $C_p \xrightarrow{a} C_{p+1}$, oblivious to symbol value; C_{2n} is the only accepting state in this set.
- ▶ Search: on reading a , $\delta(S_p, a) = \{S_{p+1}, R_{n-1}^a\}$.

Transitions

- ▶ Length-checker: $C_p \xrightarrow{a} C_{p+1}$, oblivious to symbol value; C_{2n} is the only accepting state in this set.
- ▶ Search: on reading a , $\delta(S_p, a) = \{S_{p+1}, R_{n-1}^a\}$.
- ▶ Remember: $R_k^b \xrightarrow{a} R_{k-1}^b$ (countdown, either symbol)
- ▶ at $k = 0$, compare: $R_0^b \xrightarrow{b} Matched$, $R_0^b \xrightarrow{a \neq b} Dead$.

Transitions

- ▶ Length-checker: $C_p \xrightarrow{a} C_{p+1}$, oblivious to symbol value; C_{2n} is the only accepting state in this set.
- ▶ Search: on reading a , $\delta(S_p, a) = \{S_{p+1}, R_{n-1}^a\}$.
- ▶ Remember: $R_k^b \xrightarrow{a} R_{k-1}^b$ (countdown, either symbol)
- ▶ at $k = 0$, compare: $R_0^b \xrightarrow{b} Matched$, $R_0^b \xrightarrow{a \neq b} Dead$.
- ▶ Self-loops at *Matched* and *Dead*

q_0 launches C_1, S_1 and R_{n-1}^b , and $F = \{C_{2n}, Matched\}$.

Correctness: setting up notation

- ▶ On reading the first symbol (say b), q_0 spawns 3 branches: length checker (C_1), remember (R_{n-1}^b), continue search (S_1)
- ▶ Call the run-thread rooted at a remember state R_{n-1}^b a **remember chain**
- ▶ **remember chain k** is the sequence $R_{n-1}^{w_k} \rightarrow R_{n-2}^{w_k} \rightarrow \cdots \rightarrow R_0^{w_k}$, spawned right after the k -th symbol

Correctness: setting up notation

- ▶ On reading the first symbol (say b), q_0 spawns 3 branches: length checker (C_1), remember (R_{n-1}^b), continue search (S_1)
- ▶ Call the run-thread rooted at a remember state R_{n-1}^b a **remember chain**
- ▶ **remember chain k** is the sequence $R_{n-1}^{w_k} \rightarrow R_{n-2}^{w_k} \rightarrow \cdots \rightarrow R_0^{w_k}$, spawned right after the k -th symbol
- ▶ C_1 goes till C_{2n} and accepts
- ▶ R_{n-1}^b goes till R_0^b , and if b is the next symbol, R_0^b goes to *Matched*
- ▶ S_1 goes on till S_{n-1} , ea/ch time spawning a new remember chain rooted at R_{n-1}^*

Correctness: index bookkeeping

- ▶ After reading $n - 1$ symbols: we are at states C_{n-1}, S_{n-1} , and **remember chains** $1, \dots, n - 1$

Correctness: index bookkeeping

- ▶ After reading $n - 1$ symbols: we are at states C_{n-1}, S_{n-1} , and **remember chains** $1, \dots, n - 1$
 - ▶ remember chain k , spawned right after the k -th symbol ($1 \leq k \leq n - 1$) as R_{n-1}^* has by time $n - 1$ taken $(n - 1) - k$ countdown steps, so its index is $(n - 1) - [(n - 1) - k] = k$ and is R_k^*

Correctness: index bookkeeping

- ▶ After reading $n - 1$ symbols: we are at states C_{n-1}, S_{n-1} , and **remember chains** $1, \dots, n - 1$
 - ▶ remember chain k , spawned right after the k -th symbol ($1 \leq k \leq n - 1$) as R_{n-1}^* has by time $n - 1$ taken $(n - 1) - k$ countdown steps, so its index is $(n - 1) - [(n - 1) - k] = k$ and is R_k^*
 - ▶ so remember chain 1 reaches state R_1^* after reading $n - 1$ symbols

Correctness: index bookkeeping

- ▶ After reading $n - 1$ symbols: we are at states C_{n-1}, S_{n-1} , and **remember chains** $1, \dots, n - 1$
 - ▶ remember chain k , spawned right after the k -th symbol ($1 \leq k \leq n - 1$) as R_{n-1}^* has by time $n - 1$ taken $(n - 1) - k$ countdown steps, so its index is $(n - 1) - [(n - 1) - k] = k$ and is R_k^*
 - ▶ so remember chain 1 reaches state R_1^* after reading $n - 1$ symbols
 - ▶ On reading the n th symbol, this becomes R_0^*

Correctness: index bookkeeping

- ▶ After reading $n - 1$ symbols: we are at states C_{n-1}, S_{n-1} , and **remember chains** $1, \dots, n - 1$
 - ▶ remember chain k , spawned right after the k -th symbol ($1 \leq k \leq n - 1$) as R_{n-1}^* has by time $n - 1$ taken $(n - 1) - k$ countdown steps, so its index is $(n - 1) - [(n - 1) - k] = k$ and is R_k^*
 - ▶ so remember chain 1 reaches state R_1^* after reading $n - 1$ symbols
 - ▶ On reading the n th symbol, this becomes R_0^*
 - ▶ This R_0^* checks if the $n + 1$ th symbol matches the first symbol

Correctness: index bookkeeping

- ▶ After reading $n - 1$ symbols: we are at states C_{n-1} , S_{n-1} , and **remember chains** $1, \dots, n - 1$
 - ▶ remember chain k , spawned right after the k -th symbol ($1 \leq k \leq n - 1$) as R_{n-1}^* has by time $n - 1$ taken $(n - 1) - k$ countdown steps, so its index is $(n - 1) - [(n - 1) - k] = k$ and is R_k^*
 - ▶ so remember chain 1 reaches state R_1^* after reading $n - 1$ symbols
 - ▶ On reading the n th symbol, this becomes R_0^*
 - ▶ This R_0^* checks if the $n + 1$ th symbol matches the first symbol
- ▶ On symbol n : S_{n-1} spawns the last remember chain, rooted at R_{n-1}^* ; every earlier remember chain counts down one more step;
- ▶ On symbol n : length checker reaches C_n
- ▶ From here, only n parallel remember chains (counting down, then comparing), plus the length checker

Correctness: the wrap-up

- ▶ Remember chain k takes exactly $n - 1$ countdown steps to go from R_{n-1}^* down to R_0^*

Correctness: the wrap-up

- ▶ Remember chain k takes exactly $n - 1$ countdown steps to go from R_{n-1}^* down to R_0^*
- ▶ It reaches R_0^* at time $k + (n - 1) = n + k - 1$, and compares on symbol $n + k$

Correctness: the wrap-up

- ▶ **Remember chain k** takes exactly $n - 1$ countdown steps to go from R_{n-1}^* down to R_0^*
- ▶ It reaches R_0^* at time $k + (n - 1) = n + k - 1$, and compares on symbol $n + k$
- ▶ If it achieves a symbol match, goes to *Matched*, mismatch leads to *Dead*

Correctness: the wrap-up

- ▶ **Remember chain k** takes exactly $n - 1$ countdown steps to go from R_{n-1}^* down to R_0^*
- ▶ It reaches R_0^* at time $k + (n - 1) = n + k - 1$, and compares on symbol $n + k$
- ▶ If it achieves a symbol match, goes to *Matched*, mismatch leads to *Dead*
- ▶ By time $2n$: length checker is at C_{2n} (accepting iff $|w| = 2n$ exactly), and all n remember chains have compared, chain k landing in *Matched* iff $w_k = w_{n+k}$

Correctness: the wrap-up

- ▶ **Remember chain k** takes exactly $n - 1$ countdown steps to go from R_{n-1}^* down to R_0^*
- ▶ It reaches R_0^* at time $k + (n - 1) = n + k - 1$, and compares on symbol $n + k$
- ▶ If it achieves a symbol match, goes to *Matched*, mismatch leads to *Dead*
- ▶ By time $2n$: length checker is at C_{2n} (accepting iff $|w| = 2n$ exactly), and all n remember chains have compared, chain k landing in *Matched* iff $w_k = w_{n+k}$
- ▶ **Accept** iff every reachable state is accepting: $C_{2n} \in F$ and every remember chain is at *Matched* iff the word is exactly ww for some $w \in \Sigma^n$
- ▶ One mismatched pair, or wrong length, puts *some* branch outside F

Moving on

- ▶ Existential acceptance: “some run accepts.”

Moving on

- ▶ Existential acceptance: “some run accepts.”
- ▶ Universal acceptance: “every run accepts.”

Moving on

- ▶ Existential acceptance: “some run accepts.”
- ▶ Universal acceptance: “every run accepts.”
- ▶ What if a single automaton could use *both*, at different points in the same run?

Moving on

- ▶ Existential acceptance: “some run accepts.”
- ▶ Universal acceptance: “every run accepts.”
- ▶ What if a single automaton could use *both*, at different points in the same run?
- ▶ Having both in the *same* machine, at states we choose, is the idea behind **alternation**.

Alternation

- ▶ Introduced (for Turing machines) by Chandra, Kozen, Stockmeyer, *Alternation*, J. ACM 28(1), 1981, pp. 114–133.

Alternation

- ▶ Introduced (for Turing machines) by Chandra, Kozen, Stockmeyer, *Alternation*, J. ACM 28(1), 1981, pp. 114–133.
- ▶ Built on two independent 1976 FOCS papers : one by Kozen, one by Chandra-Stockmeyer later merged into the joint journal version.

Alternation

- ▶ Introduced (for Turing machines) by Chandra, Kozen, Stockmeyer, *Alternation*, J. ACM 28(1), 1981, pp. 114–133.
- ▶ Built on two independent 1976 FOCS papers : one by Kozen, one by Chandra-Stockmeyer later merged into the joint journal version.
- ▶ States are $\exists$ or $\forall$; a computation *alternates* between them, generalizing nondeterminism (which only has $\exists$).

Alternation

- ▶ Introduced (for Turing machines) by Chandra, Kozen, Stockmeyer, *Alternation*, J. ACM 28(1), 1981, pp. 114–133.
- ▶ Built on two independent 1976 FOCS papers : one by Kozen, one by Chandra-Stockmeyer later merged into the joint journal version.
- ▶ States are $\exists$ or $\forall$; a computation *alternates* between them, generalizing nondeterminism (which only has $\exists$).
- ▶ Landmark results connecting **space and time**:
 - ▶ alternating LOGSPACE = PTIME;
 - ▶ alternating PSPACE = EXPTIME
 - ▶ alternating EXPSPACE = 2EXPTIME ...
- ▶ They say: a machine that never gets to use $\forall$ needs exponentially more of one resource than an alternating machine needs of the other, for the same problem.

In our setting of L_n

- ▶ L_n demonstrates the trade-off :

In our setting of L_n

- ▶ L_n demonstrates the trade-off :
 - ▶ *Angelic NFA* (2^n states, $\exists$ -only) existentially guesses-and-remembers all of w as one joint piece of memory, and pays exponential cost.

In our setting of L_n

- ▶ L_n demonstrates the trade-off :
 - ▶ *Angelic NFA* (2^n states, $\exists$ -only) existentially guesses-and-remembers all of w as one joint piece of memory, and pays exponential cost.
 - ▶ *Devilish* ($O(n)$ states, has $\forall$) splits into n independent position-checks, reusing the same $O(n)$ -state pool across all of them. The cost only collapses once $\forall$ is added.

Alternating finite automata (AFA)

- ▶ Same 1981 Chandra-Kozen-Stockmeyer paper specializes to finite automata: **alternating finite automata (AFAs)**.

Alternating finite automata (AFA)

- ▶ Same 1981 Chandra-Kozen-Stockmeyer paper specializes to finite automata: **alternating finite automata (AFAs)**.
- ▶ States split into $\exists$ -states (nondeterministic choice) and $\forall$ -states (spawn all successors, all must accept)

$\forall\exists$ Automata

- ▶ A $\forall\exists$ automaton is a tuple $(Q, \Sigma, \Delta, S, F)$ where

$\forall\exists$ Automata

- ▶ A $\forall\exists$ automaton is a tuple $(Q, \Sigma, \Delta, S, F)$ where
- ▶ Q is partitioned into $Q_{\forall}$ and $Q_{\exists}$,

$\forall\exists$ Automata

- ▶ A $\forall\exists$ automaton is a tuple $(Q, \Sigma, \Delta, S, F)$ where
- ▶ Q is partitioned into $Q_{\forall}$ and $Q_{\exists}$,
- ▶ Transitions out of a $Q_{\exists}$ state are “or” transitions,
- ▶ Transitions out of a $Q_{\forall}$ state are “and” transitions

“Or” and “And” transitions

- ▶ “Or” transition ($q \in Q_{\exists}$):

$$\Delta(q, a) = q_{i_1} \vee \cdots \vee q_{i_j}$$

“Or” and “And” transitions

- ▶ “Or” transition ($q \in Q_{\exists}$):

$$\Delta(q, a) = q_{i_1} \vee \cdots \vee q_{i_j}$$

- ▶ “And” transition ($q \in Q_{\forall}$):

$$\Delta(q, a) = q_{j_1} \wedge \cdots \wedge q_{j_l}$$

“Or” and “And” transitions

- ▶ “Or” transition ($q \in Q_{\exists}$):

$$\Delta(q, a) = q_{i_1} \vee \cdots \vee q_{i_j}$$

- ▶ “And” transition ($q \in Q_{\forall}$):

$$\Delta(q, a) = q_{j_1} \wedge \cdots \wedge q_{j_l}$$

- ▶ An “and” transition spawns *all* of $q_{j_1}, \dots, q_{j_l}$ as parallel threads.
- ▶ An “or” transition is ordinary nondeterministic choice : pick one.

Example

► $Q = \{s_1, s_2, s_3, s_4, s_5\}$, $S = \{s_1\}$, $F = \{s_5\}$.

Example

- ▶ $Q = \{s_1, s_2, s_3, s_4, s_5\}$, $S = \{s_1\}$, $F = \{s_5\}$.
- ▶ $\Delta(s_1, a) = s_2 \vee s_3$
- ▶ $\Delta(s_2, a) = s_2 \wedge s_4$, $\Delta(s_2, b) = s_5$
- ▶ $\Delta(s_4, a) = s_4$, $\Delta(s_4, b) = s_5$
- ▶ $\Delta(s_3, a) = s_3$, $\Delta(s_3, b) = s_3$

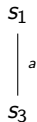
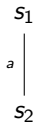
Two run trees on $w = aab$

$$\begin{array}{lll} \delta(s_1, a) = s_2 \vee s_3 & \delta(s_2, a) = s_2 \wedge s_4 & \delta(s_2, b) = s_5 \\ \delta(s_4, b) = s_5 & \delta(s_3, a) = s_3 & \delta(s_3, b) = s_3 \end{array}$$

s_1

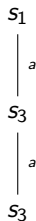
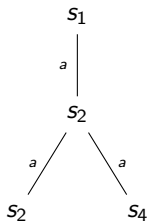
Two run trees on $w = aab$

$$\begin{array}{lll} \delta(s_1, a) = s_2 \vee s_3 & \delta(s_2, a) = s_2 \wedge s_4 & \delta(s_2, b) = s_5 \\ \delta(s_4, b) = s_5 & \delta(s_3, a) = s_3 & \delta(s_3, b) = s_3 \end{array}$$



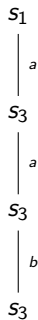
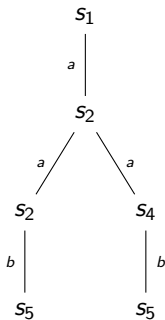
Two run trees on $w = aab$

$$\begin{array}{lll} \delta(s_1, a) = s_2 \vee s_3 & \delta(s_2, a) = s_2 \wedge s_4 & \delta(s_2, b) = s_5 \\ \delta(s_4, b) = s_5 & \delta(s_3, a) = s_3 & \delta(s_3, b) = s_3 \end{array}$$



Two run trees on $w = aab$

$$\begin{aligned}\delta(s_1, a) &= s_2 \vee s_3 & \delta(s_2, a) &= s_2 \wedge s_4 & \delta(s_2, b) &= s_5 \\ \delta(s_4, b) &= s_5 & \delta(s_3, a) &= s_3 & \delta(s_3, b) &= s_3\end{aligned}$$



Acceptance for a run tree

- ▶ Each “and” step spawns multiple threads, evolving in parallel.

Acceptance for a run tree

- ▶ Each “and” step spawns multiple threads, evolving in parallel.
- ▶ A word w is accepted *by a run tree* if a final state is reached in *every* thread once w is fully read.

Acceptance for a run tree

- ▶ Each “and” step spawns multiple threads, evolving in parallel.
- ▶ A word w is accepted *by a run tree* if a final state is reached in *every* thread once w is fully read.
- ▶ Left tree: both leaves are $s_5 \in F$ **accepting**.
- ▶ Right tree: leaf is $s_3 \notin F$ **not accepting**.

Acceptance for the automaton

- ▶ An “or” transition is ordinary nondeterminism: many different run trees are possible (one per choice made at each $\exists$ state).

Acceptance for the automaton

- ▶ An “or” transition is ordinary nondeterminism: many different run trees are possible (one per choice made at each $\exists$ state).
- ▶ w is accepted by the automaton if *at least one* run tree on w is accepting.

Acceptance for the automaton

- ▶ An “or” transition is ordinary nondeterminism: many different run trees are possible (one per choice made at each $\exists$ state).
- ▶ w is accepted by the automaton if *at least one* run tree on w is accepting.
- ▶ Here: the left run tree already accepts aab , so $aab \in L(A)$.

Expressiveness vs. NFA

- ▶ How does the class of languages recognized by $\forall\exists$ automata compare to NFA-recognizable (regular) languages?

Expressiveness vs. NFA

- ▶ How does the class of languages recognized by $\forall\exists$ automata compare to NFA-recognizable (regular) languages?
- ▶ Can every NFA be simulated by a $\forall\exists$ automaton?
- ▶ Can every $\forall\exists$ automaton be simulated by an NFA?

Complementing a $\forall\exists$ automaton

- ▶ Complementing an NFA usually needs determinization first (subset construction), then flipping accept/reject.

Complementing a $\forall\exists$ automaton

- ▶ Complementing an NFA usually needs determinization first (subset construction), then flipping accept/reject.
- ▶ Does a $\forall\exists$ automaton admit a *direct* complementation (like DFAs)?